Федеральное государственное автономное образовательное учреждение
высшего образования

«Национальный исследовательский технологический университет
«МИСИС»

Институт компьютерных наук (ИКН)

Кафедра Инфокоммуникационных технологий (ИКТ)

Лабораторная работа №2

по дисциплине «Технологии программирования»

на тему «Основы ООП»

Выполнил:
студент группы БИВТ-25-4-5
Марьин Артемий Валерьевич

Москва, 2026

Цель работы: изучить на практике основные принципы объектно-ориентированного программирования (абстракцию, инкапсуляцию, наследование и полиморфизм), освоить различие между абстрактным классом и интерфейсом и построить на языке C# иерархию базовых классов для пошаговой стратегии.

1. Ход выполнения работы:

1.1. Проектирование иерархии:

Необходимо реализовать иерархию классов для пошаговой стратегии. Корнем иерархии является класс GameObject, хранящий идентификатор, имя и координаты объекта. От него наследуются два класса: Unit и Building. Конкретные классы: Archer наследует Unit, а Fort и MobileHome от Building. Способности «атаковать» и «двигаться» вынесены в интерфейсы IAttacker и IMoveable. IAttacker реализуют Archer и Fort, IMoveable – Archer и MobileHome. Названия интерфейсов начинаются с префикса I в соответствии с соглашением об именовании в C#.

Класс GameObject объявлен абстрактным, так как в игре не существует «просто игрового объекта», есть только юниты и постройки. Классы Unit и Building – неабстрактные. Они описывают полноценные сущности с собственным состоянием и поведением (здоровье и признак завершённости строительства), поэтому их экземпляры можно использовать как базовые тестовые объекты. При этом такие объекты сами по себе не умеют ни атаковать, ни двигаться.

Способности атаковать и двигаться реализованы интерфейсами, а не наследованием, потому что они пересекаются между ветками иерархии: Archer (ветка Unit) и Fort (ветка Building) оба атакуют, а Archer и MobileHome оба двигаются. В C# у класса может быть только один родитель, но реализовывать можно любое количество интерфейсов, поэтому Archer одновременно является Unit, IAttacker и IMoveable.

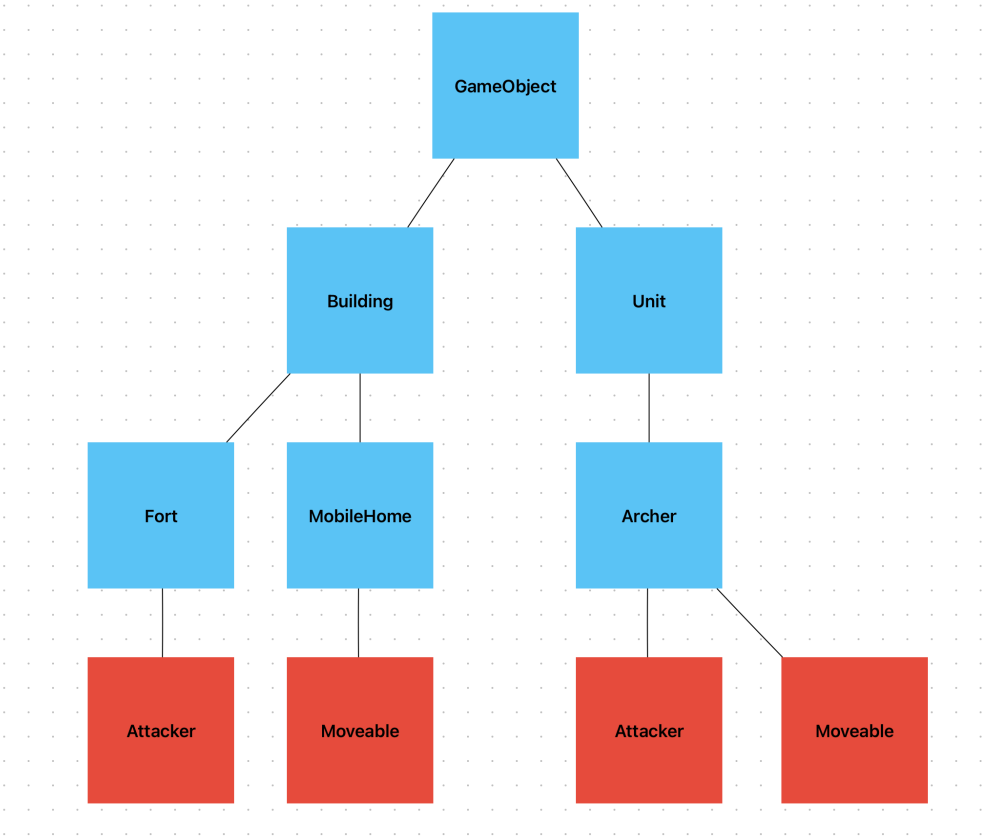


Рис. 1 – Диаграмма иерархии классов

1.2. Создание проекта и структура решения:

Проект создан как консольное приложение на языке C# для платформы .NET 10; в файле проекта включены неявные using-директивы и анализ допустимости значений null (Nullable). Исходный код разделён по пространствам имён: Game.Interfaces, Game.Objects, Game.Objects.Units и Game.Objects.Buildings. Точкой входа служит файл Program.cs с операторами верхнего уровня.

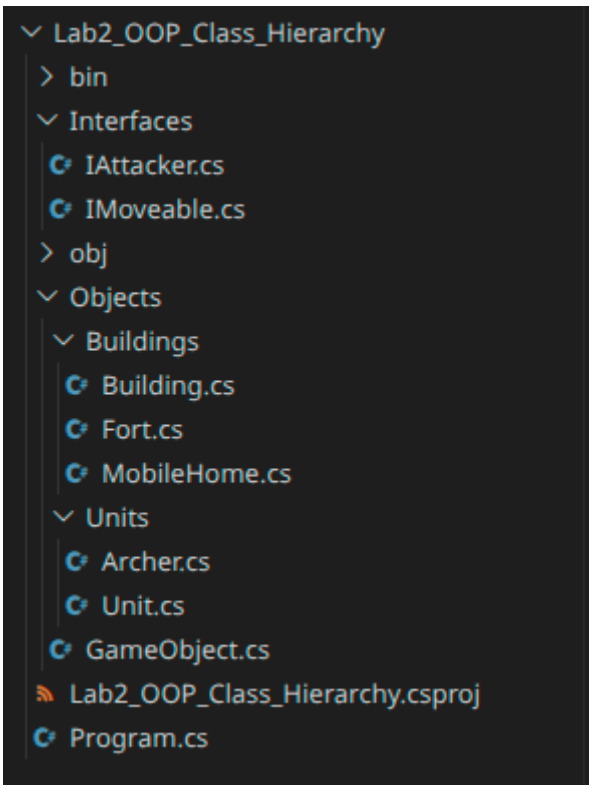


Рис. 2 – Структура проекта

1.3. Реализация интерфейсов IAttacker и IMoveable:

Интерфейс IAttacker содержит единственный метод Attack(Unit unit) – контракт «умею атаковать юнит». Интерфейс IMoveable содержит метод Move(int x, int y) – контракт «умею перемещаться в заданную точку». Интерфейсы не содержат состояния и описывают только роль объекта; благодаря малому размеру каждый класс реализует лишь те способности, которые ему действительно нужны. Файлы IAttacker.cs и IMoveable.cs:

```
namespace Game.Interfaces;

using Game.Objects.Units;

public interface IAttacker

{

void Attack (Unit unit);

}
```

Листинг 1 – IAttacker.cs

```
namespace Game.Interfaces;

public interface IMoveable{

void Move(int x, int y);

}
```

Листинг 2 – IMoveable.cs

1.4. Реализация базового класса GameObject:

GameObject – абстрактный базовый класс всех игровых объектов. Идентификатор и имя хранятся в приватных полях только для чтения, координаты хранятся в приватных полях; снаружи данные доступны через геттеры GetId(), GetName(), GetX() и GetY(). Конструктор защищённый, проверяет, что имя не пустое.

Для изменения координат предусмотрен защищённый метод SetPosition(): наследники не обращаются к полям напрямую, а меняют положение через единую точку входа. Решение о том, разрешено ли перемещение (жив ли юнит, достроено ли здание), принимается в методах Move() конкретных классов, а само изменение состояния выполняет базовый класс. Файл GameObject.cs:

```
namespace Game.Objects;

public abstract class GameObject

{

private readonly int _id;

private readonly string _name;

private int _x;

private int _y;

protected GameObject(int id, string name, int x, int y)

{

if (string.IsNullOrWhiteSpace(name))

throw new ArgumentException("Name must not be empty", nameof(name));

_id = id;

_name = name;

_x = x;

_y = y;

}

public int GetId() => _id;

public string GetName() => _name;

public int GetX() => _x;

public int GetY() => _y;

protected void SetPosition(int x, int y)
```

```
{
_x = x;
_y = y;
}
}
```

Листинг 3 – GameObject.cs

1.5. Реализация класса Unit:

Класс Unit наследует GameObject и описывает юнита со здоровьем. Здоровье хранится в приватном поле _hp; конструктор вызывает базовый конструктор и проверяет, что значение не отрицательное. Метод IsAlive() возвращает true, пока здоровье больше нуля, а GetHp() возвращает текущее здоровье. Метод ReceiveDamage(float damage) отклоняет отрицательный урон и не позволяет здоровью опуститься ниже нуля. Изменить здоровье можно только через этот метод. Файл Unit.cs:

```
namespace Game.Objects.Units;
public class Unit : GameObject
{
private float _hp;
public Unit(int id, string name, int x, int y, float hp) : base(id, name, x, y)
{
if (hp < 0)
{
throw new ArgumentOutOfRangeException(nameof(hp), "HP value must not be negative");
}
_hp = hp;
}
public bool IsAlive() => _hp > 0;
public float GetHp() => _hp;
public void ReceiveDamage(float damage)
{
if (damage < 0)
throw new ArgumentOutOfRangeException(nameof(damage), "Damage value must not be negative");
_hp = Math.Max(0, _hp - damage);
}
```

```
}
```

Листинг 4 – Unit.cs

1.6. Реализация класса Archer:

Класс Archer наследует Unit и реализует интерфейсы IAttacker и IMoveable. Урон лучника задаётся в конструкторе и проверяется на неотрицательность. Метод Attack() проверяет аргумент цель на предмет null. Если атакующий жив, то цели наносится урон через ReceiveDamage(). Метод Move() для живого лучника вызывает SetPosition(), а для мёртвого ничего не делает. Файл Archer.cs:

```
namespace Game.Objects.Units;

using Game.Interfaces;

public class Archer : Unit, IAttacker, IMoveable

{

private float _damage;

public Archer(int id, string name, int x, int y, float hp, float damage) : base(id, name, x, y, hp)

{

if (damage < 0)

throw new ArgumentOutOfRangeException(nameof(damage), "Damage value must not be negative");

_damage = damage;

}

public void Attack(Unit unit)

{

if (unit is null)

throw new ArgumentNullException(nameof(unit));

if (!IsAlive()) return;

unit.ReceiveDamage(_damage);

}

public void Move(int x, int y)

{

if (!IsAlive()) return;

SetPosition(x, y);

}

}
```

Листинг 5 – Archer.cs

1.7. Реализация класса Building:

Класс Building наследует GameObject и описывает постройку. Признак завершённости хранится в приватном поле _isBuilt; параметр isBuilt конструктора по умолчанию равен false (новая постройка считается недостроенной). Метод IsBuilt() возвращает этот признак, а метод Build() помечает постройку как построенную. Файл Building.cs:

```
namespace Game.Objects.Buildings;

public class Building : GameObject

{

private bool _isBuilt;

public Building(int id, string name, int x, int y, bool isBuilt = false) : base(id, name, x, y)

{

_isBuilt = isBuilt;

}

public bool IsBuilt() => _isBuilt;

public void Build()

{

_isBuilt = true;

}

}
```

Листинг 6 – Building.cs

1.8. Реализация классов Fort и MobileHome:

Класс Fort наследует Building и реализует IAttacker: после проверки аргумента на null и статуса постройки крепости он наносит урон цели. Класс MobileHome наследует Building и реализует IMoveable: перемещается только после завершения строительства. Файлы Fort.cs и MobileHome.cs:

```
namespace Game.Objects.Buildings;

using Game.Interfaces;

using Game.Objects.Units;

public class Fort : Building, IAttacker

{

private float _damage;

public Fort(int id, string name, int x, int y, float damage, bool isBuilt = false) : base(id, name, x, y, isBuilt)
```

```
{
if (damage < 0)
throw new ArgumentOutOfRangeException(nameof(damage), "Damage value must not be negative");
_damage = damage;
}
public void Attack(Unit unit)
{
if (unit is null)
throw new ArgumentNullException(nameof(unit));
if (!IsBuilt()) return;
unit.ReceiveDamage(_damage);
}
}
```

Листинг 7 – Fort.cs

```
namespace Game.Objects.Buildings;
using Game.Interfaces;
public class MobileHome : Building, IMoveable
{
public MobileHome(int id, string name, int x, int y, bool isBuilt = false) : base(id, name, x, y, isBuilt)
{
}
public void Move(int x, int y)
{
if (!IsBuilt()) return;
SetPosition(x, y);
}
}
```

Листинг 8 – MobileHome.cs

1.9. Демонстрация работы иерархии классов:

Для проверки иерархии написан сценарий в файле Program.cs. Создаются два лучника, крепость и дом на колёсах, после чего последовательно проверяется: перемещение лучника; уничтожение одного лучника другим; невозможность для погибшего лучника двигаться и атаковать, отказ недостроенной крепости стрелять и успешная стрельба после вызова

Build(), отказ недостроенного дома двигаться и успешное перемещение после достройки. Для вывода состояния объектов используются вспомогательные функции PrintObject() и PrintUnit(). Файл Program.cs:

```
using Game.Objects;
using Game.Objects.Units;
using Game.Objects.Buildings;
//var testUnit = new Unit(-1,"",1,0, -2);
var arch1 = new Archer(id: 1, name: "arch1", x: 0, y: 0, hp: 100f, damage: 25f);
var arch2 = new Archer(id: 2, name: "arch2", x: 8, y: 8, hp: 30f, damage: 10f);
var fort = new Fort(id: 4, name: "keep", x: 10, y: 10, damage: 40f);
var wagon = new MobileHome(id: 5, name: "wagon", x: 2, y: 2);
Console.WriteLine("Archer moves");
PrintObject(arch1);
arch1.Move(3, 4); //archer move test
PrintObject(arch1);
Console.WriteLine("Archer1 attacks and kills Archer2");
PrintUnit(arch2);
while (arch2.IsAlive())
{
arch1.Attack(arch2); //arch1 attack test
PrintUnit(arch2); //arch1 kills arch2
}
Console.WriteLine("We try to move Archer2");
Console.WriteLine($"{arch2.GetName()} (dead) tries to move to (0, 0)");
arch2.Move(0, 0); //(dead) arch2 tries to move
PrintUnit(arch2);
Console.WriteLine("We try to force Archer2 to attack Archer1");
PrintUnit(arch1);
Console.WriteLine($"{arch2.GetName()} (dead) tries to attack {arch1.GetName()}");
arch2.Attack(arch1); //(dead) arch2 tries to attack arch1
PrintUnit(arch1);
Console.WriteLine("Fort attacks only after it is built");
Console.WriteLine($"{fort.GetName()} built: {fort.IsBuilt()}"); //Fort attacks only after it is built
fort.Attack(arch1); //unbuilt fort fails
Console.WriteLine("Unbuilt fort fires at arch1");
```

```
PrintUnit(arch1);
fort.Build();
Console.WriteLine($"{fort.GetName()} built: {fort.IsBuilt()}");
fort.Attack(arch1); //built fort succeeds
Console.WriteLine("Built fort fires at arch1");
PrintUnit(arch1);
Console.WriteLine("Mobile home moves only after it is built");
Console.WriteLine($"{wagon.GetName()} built: {wagon.IsBuilt()}");
PrintObject(wagon);
wagon.Move(7, 3);
Console.WriteLine("Unbuilt wagon tries to move to (7, 3):");
PrintObject(wagon);
wagon.Build();
wagon.Move(7, 3);
Console.WriteLine("Built wagon moves to (7, 3):");
PrintObject(wagon);
//Print functions
static void PrintObject(GameObject obj)
{
Console.WriteLine($" [{obj.GetId()}] {obj.GetName()} at ({obj.GetX()}, {obj.GetY()})");
}
static void PrintUnit(Unit unit)
{
Console.WriteLine(
$" [{unit.GetId()}] {unit.GetName()} at ({unit.GetX()}, {unit.GetY()}), " +
$"HP: {unit.GetHp()}, alive: {unit.IsAlive()}");
}
```

Листинг 9 – Program.cs: сценарий демонстрации иерархии классов

2. Результаты работы:

2.1. Запуск программы:

Результат выполнения программы приведён на рис. 3: лучник перемещается и уничтожает противника; погибший лучник не двигается и не атакует; недостроенная крепость не стреляет, а построенная наносит

урон; недостроенный дом на колёсах не едет, а построенный перемещается в заданную точку.

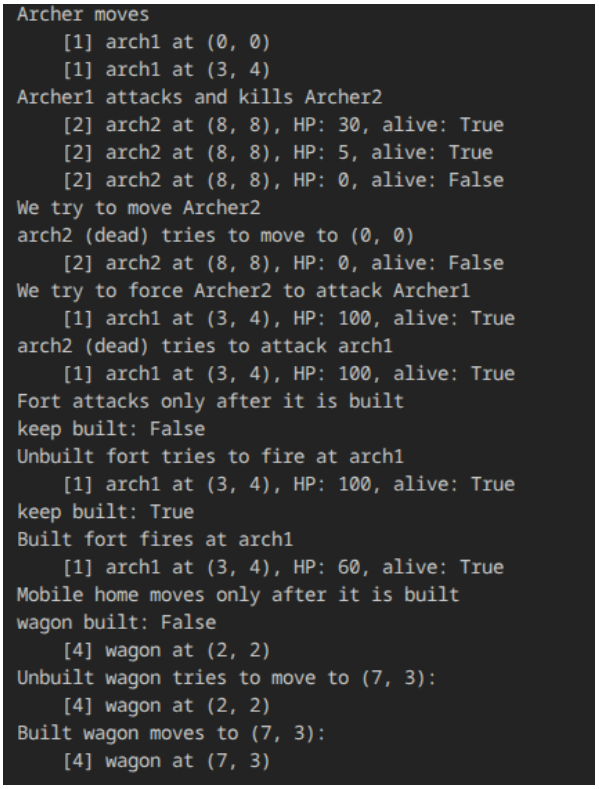


Рис. 3 – Окно консоли с результатом работы программы

2.2. Публикация проекта в удалённом репозитории:

Проект опубликован в удалённом репозитории https://github.com/MarinArtemiy/labs_TP.git. Была выполнена команда: git init, после чего все файлы добавлены в индекс командой git add . и сохранены первым коммитом. Локального репозиторий был связан с удалённым репозиторием командой: git remote add origin https://github.com/MarinArtemiy/labs_TP.git.